

OpenManus + Ollama

Guia de Instalacion

Agente de IA autonomo, gratuito y local
para Windows 11 - Sin Docker

Coste total: 0 euros

Modelo local con Ollama - Sin API keys

Instalacion directa con Python + pip

Windows 11

16 GB RAM

Python 3.11

Ollama + qwen2.5:7b

Contenido

1. Que es OpenManus y por que esta guía	3
2. Por que OpenManus y no otro	3
3. Requisitos previos	4
3.1 Requisitos de hardware	4
3.2 Software que necesitas instalar	4
4. Instalacion paso a paso	5
4.1 Fase 1: Instalar Python 3.11 o 3.12	5
4.2 Fase 2: Instalar Ollama y el modelo de IA	5
4.3 Fase 3: Instalar Git	6
4.4 Fase 4: Descargar e instalar OpenManus	6
5. Ejecutar OpenManus	8
5.1 Modos de ejecucion	8
5.2 Primera ejecucion y prueba	8
6. Solucion de problemas frecuentes	9
6.1 Error: "python no se reconoce como comando"	9
6.2 Error: "ollama no se reconoce como comando"	9
6.3 Error: "No module named xxx"	9
6.4 Error: "Connection refused" al conectar con Ollama	10
6.5 El modelo responde muy lento	10
6.6 Error en browser_use o navegacion web	10

7. Modelo alternativo: Agent-Zero sin Docker	10
7.1 Instalacion rapida de Agent-Zero	10
8. Resumen de todos los comandos	11
8.1 Instalacion completa (solo la primera vez)	11
8.2 Ejecutar OpenManus (cada vez que lo uses)	12
8.3 Actualizar OpenManus (periodicamente)	12

1. Que es OpenManus y por que esta guia

OpenManus es un agente de IA autonomo de codigo abierto creado por el equipo de FoundationAgents (anteriormente conocido como MetaGPT). Es, en esencia, un clon directo de Manus AI: le das un objetivo en lenguaje natural y el agente planifica, navega la web, escribe codigo, crea archivos y entrega resultados de forma autnoma. La diferencia fundamental con Manus es que OpenManus es completamente gratuito, de codigo abierto, y puedes ejecutarlo en tu propio ordenador sin depender de servidores ajenos ni de creditos de pago.

Esta guia esta disenada especificamente para instalar OpenManus en Windows 11 **sin necesidad de Docker**. A diferencia de otros agentes como AgenticSeek o Suna que requieren Docker para funcionar, OpenManus se instala directamente con Python y pip, como cualquier programa normal de Windows. No necesitas WSL2, no necesitas maquinas virtuales, no necesitas particiones de Linux. Todo funciona nativamente en tu sistema Windows.

Ademas, esta guia utiliza **Ollama** como motor de IA local, lo que significa que no necesitas pagar ninguna API key de OpenAI, Anthropic ni Google. El modelo de IA corre en tu propio ordenador, usando tu RAM y tu CPU. El coste total de la instalacion es de cero euros. Tu unico requisito es tener 16 GB de RAM, que es lo que tiene tu equipo.

2. Por que OpenManus y no otro

Existen varias alternativas de codigo abierto a Manus AI, pero la gran mayoria requiere Docker para funcionar. A continuacion se presenta una comparativa clara de las opciones mas conocidas, destacando si necesitan Docker o no, si funcionan con modelos locales gratuitos, y su nivel de dificultad de instalacion en Windows 11. Esta tabla te ayudara a entender por que OpenManus es la mejor opcion para tu caso concreto.

Proyecto	Docker	Modelo local	Dificultad	Estado
OpenManus	NO	Si (Ollama)	Facil	Estable
AgenticSeek	SI	Si (Ollama)	Media	Beta
Suna	SI	No	Alta	Beta
AutoGPT	SI	Parcial	Alta	Estable
OpenHands	SI	Parcial	Alta	Estable
Agent-Zero	Opcional	Si (Ollama)	Media	Beta

Tabla 1: Comparativa de alternativas open-source a Manus AI

Como puedes ver, OpenManus es el unico que combina las tres ventajas clave: no necesita Docker, funciona con modelos locales gratuitos via Ollama, y tiene un nivel de dificultad de instalacion bajo. Agent-Zero permite instalacion sin Docker, pero su configuracion es mas compleja y su funcionamiento es menos similar a Manus. Por todo ello, esta guia se centra en OpenManus.

3. Requisitos previos

Antes de comenzar la instalacion, necesitas asegurarte de que tu equipo cumple con los requisitos minimos y tener descargados los programas base. Tu equipo con Windows 11 y 16 GB de RAM cumple sobradamente con todo lo necesario. A continuacion se detallan los requisitos de hardware y software que necesitas antes de empezar con el proceso de instalacion paso a paso.

3.1 Requisitos de hardware

Componente	Minimo	Tu equipo
RAM	8 GB	16 GB
CPU	4 nucleos	Cumple
Disco	10 GB libres	Verifica
GPU	Opcional	Verifica
SO	Windows 10/11	Windows 11

Tabla 2: Requisitos de hardware

La GPU es opcional pero recomendable. Si tienes una tarjeta grafica dedicada NVIDIA o AMD, el modelo de IA corrrera mucho mas rapido. Para comprobarlo, abre el Administrador de tareas (Ctrl+Shift+Esc), ve a la pestana "Rendimiento" y mira la seccion "GPU". Si ves "NVIDIA" o "AMD" junto a un modelo especifico, tienes GPU dedicada. Si solo ves "Intel" o "GPU 0 - Intel UHD", es integrada y no acelerara el modelo, pero aun asi funcionara usando la CPU.

3.2 Software que necesitas instalar

Necesitas instalar cuatro programas antes de proceder con OpenManus. Todos son instaladores normales de Windows (doble clic y siguiente). No necesitas Linux, ni WSL2, ni Docker. A continuacion se listan los cuatro programas con sus enlaces de descarga directos.

Programa	Version	Descarga	Para que sirve
Python	3.11.x	python.org/downloads	Ejecutar OpenManus
Ollama	Ultima	ollama.com/download	Motor de IA local
Git	Ultima	git-scm.com/download/win	Descargar OpenManus
uv (opcional)	Ultima	docs.astral.sh/uv	Instalacion rapida

Tabla 3: Software necesario

4. Instalacion paso a paso

El proceso de instalacion se divide en cuatro fases claras. Cada fase depende de la anterior, así que debes seguirlas en orden. No te saltes ningun paso. Al finalizar, tendras OpenManus funcionando con un modelo de IA local completamente gratuito, sin necesidad de Docker ni de pagar API keys de ningun tipo.

4.1 Fase 1: Instalar Python 3.11 o 3.12

Python es el lenguaje de programacion en el que esta escrito OpenManus. Sin Python, no puedes ejecutarlo. Las versiones recomendadas son **Python 3.11** o **Python 3.12**. Ambas son compatibles con OpenManus. Si ya tienes una de estas versiones instalada, no necesitas reinstalar. Evita Python 3.13, ya que algunas dependencias de OpenManus aun no son compatibles con esa version.

Pasos:

1. Ve a python.org/downloads y descarga Python 3.12.x (o 3.11.x) para Windows (el instalador de 64 bits)
2. Ejecuta el instalador
3. **MUY IMPORTANTE:** Marca la casilla "Add Python to PATH" en la primera pantalla del instalador. Si no lo haces, los comandos de Python no funcionaran en la terminal
4. Haz clic en "Install Now" y espera a que termine
5. Para verificar, abre una terminal nueva (cmd o PowerShell) y escribe:

```
python --version
```

Deberia mostrar algo como "Python 3.12.x" o "Python 3.11.x". Si muestra un error, probablemente olvidaste marcar "Add to PATH". Desinstala y vuelve a instalar marcando esa casilla.

4.2 Fase 2: Instalar Ollama y el modelo de IA

Ollama es el motor que ejecuta el modelo de inteligencia artificial en tu propio ordenador, sin necesidad de internet ni de pagar API keys. Piensa en Ollama como un "reproductor" de modelos de IA: tu le das un modelo y el lo ejecuta localmente usando la RAM y CPU de tu equipo. Con 16 GB de RAM, el modelo recomendado es **qwen2.5:7b**, que ofrece un buen equilibrio entre inteligencia y consumo de recursos. Este modelo soporta el uso de herramientas (tool calling), que es fundamental para que OpenManus funcione correctamente como agente autonomo.

Pasos:

1. Ve a ollama.com/download/windows y descarga el instalador para Windows
2. Ejecuta el instalador y sigue los pasos por defecto
3. Abre una terminal nueva y verifica con:

```
ollama --version
```

4. Descarga el modelo de IA (esto tardara entre 5 y 15 minutos dependiendo de tu conexion):

```
ollama pull qwen2.5:7b
```

5. El modelo ocupa unos 4.7 GB. Se descargara y quedara almacenado en tu disco

6. Verifica que funciona con:

```
ollama list
```

Deberias ver "qwen2.5:7b" en la lista. Si lo ves, el modelo esta listo para usarse.

Nota sobre modelos alternativos: Existen varias opciones segun tu equipo. **qwen2.5:7b** es el punto dulce para 16 GB de RAM. Si va lento, usa **qwen2.5:3b** (mas rapido, menos inteligente). Si tienes GPU NVIDIA con 8 GB+ de VRAM, usa **qwen2.5:14b** (mas inteligente). Tambien puedes probar **qwen3:8b**, un modelo mas reciente con mejor soporte de tool calling, aunque tiene problemas conocidos de alucinacion con Ollama (falla en torno al 80% de las llamadas a herramientas). Si qwen3:8b te da errores, vuelve a qwen2.5:7b que es mas estable.

Advertencia sobre qwen2.5-coder: No uses el modelo "qwen2.5-coder" con OpenManus. Aunque suena tentador para un agente que escribe codigo, qwen2.5-coder NO soporta correctamente tool calling (llamadas a herramientas), lo que significa que OpenManus no podra usar sus herramientas de navegacion web, ejecucion de codigo, ni creacion de archivos. Usa siempre "qwen2.5" (sin el sufijo -coder) o "qwen3".

4.3 Fase 3: Instalar Git

Git es la herramienta que te permite descargar el codigo fuente de OpenManus desde GitHub. Es un programa estandar que se instala como cualquier otro en Windows. No necesitas saber usar Git para esta guía, solo necesitas el comando "git clone" para descargar OpenManus.

Pasos:

1. Ve a git-scm.com/download/win y descarga el instalador
2. Ejecuta el instalador con las opciones por defecto (no cambies nada)
3. Verifica en una terminal nueva con:

```
git --version
```

Deberia mostrar la version instalada de Git.

4.4 Fase 4: Descargar e instalar OpenManus

Esta es la fase principal donde descargamos y configuramos OpenManus. El proceso consiste en clonar el repositorio de GitHub, instalar las dependencias de Python, y configurar el archivo de conexion con Ollama. Sigue cada paso con cuidado y no te saltes ninguno.

Paso 1: Descargar OpenManus

Abre una terminal (cmd o PowerShell) y escribe estos comandos uno por uno:

```
cd %USERPROFILE%
git clone https://github.com/FoundationAgents/OpenManus.git
cd OpenManus
```

Esto descarga el codigo de OpenManus en una carpeta llamada "OpenManus" dentro de tu carpeta de usuario (C:\Users\TuNombre\OpenManus).

Paso 2: Crear un entorno virtual de Python

Un entorno virtual es una carpeta aislada donde se instalan las dependencias de Python de OpenManus sin afectar al resto de tu sistema. Es una buena practica que evita conflictos entre proyectos. Crea el entorno con estos comandos:

```
python -m venv venv
venv\Scripts\activate
```

Despues de activar el entorno, veras "(venv)" al principio de la linea de comandos. Esto confirma que el entorno esta activo.

Paso 3: Instalar las dependencias

Con el entorno virtual activado, instala las dependencias necesarias. Hay dos metodos:

Metodo A (recomendado) - Usando uv:

```
pip install uv
uv pip install -r requirements.txt
```

Metodo B - Usando pip directamente:

```
pip install -r requirements.txt
```

El Metodo A con uv es mas rapido (puede ser 10 veces mas veloz que pip). Si prefieres lo sencillo, usa el Metodo B. Ambos instalan exactamente las mismas dependencias. La instalacion puede tardar entre 2 y 10 minutos dependiendo de tu conexion a internet.

Paso 4: Configurar Ollama como motor de IA

Este es el paso mas importante. Necesitas crear un archivo de configuracion para que OpenManus se conecte con Ollama en lugar de con OpenAI o Anthropic. OpenManus ya incluye un archivo de ejemplo para Ollama que solo necesitas copiar y activar. Los comandos son:

```
cd config
copy config.example-model-ollama.toml config.toml
cd ..
```

Ahora abre el archivo config.toml con el Bloc de notas y verifica que contiene algo como esto:

```
[llm]
api_type = "ollama"
model = "qwen2.5:7b"
base_url = "http://localhost:11434/v1"
```

Si el modelo en el archivo no coincide con el que descargaste, cambialo. El campo "model" debe ser exactamente el nombre del modelo que descargaste con "ollama pull". Si usaste qwen2.5:3b, pon eso. Guarda el archivo y cierra el Bloc de notas.

Importante: Ollama debe estar ejecutandose en segundo plano para que OpenManus pueda conectarse. Despues de instalar Ollama, este se ejecuta automaticamente como un programa en segundo plano. Puedes verificarlo abriendo el Administrador de tareas y buscando "ollama" en la lista de procesos. Si no esta, abre una terminal y escribe "ollama serve" para iniciarlo.

5. Ejecutar OpenManus

Una vez completadas las cuatro fases de instalacion, ya puedes ejecutar OpenManus. El proceso es sencillo: abre una terminal, navega a la carpeta de OpenManus, activa el entorno virtual, y ejecuta el programa. A continuacion se detallan los pasos exactos.

5.1 Modos de ejecucion

OpenManus ofrece tres modos de ejecucion diferentes. El modo basico funciona en la terminal, el modo web te da una interfaz grafica en el navegador, y el modo MCP anade herramientas adicionales. Para empezar, te recomiendo el modo web, que es el mas facil de usar. A continuacion se describen los tres modos disponibles.

Modo 1: Interfaz de terminal (basico)

Abre una terminal (cmd o PowerShell) y ejecuta:

```
cd %USERPROFILE%\OpenManus
venv\Scripts\activate
python main.py
```

Despues de ejecutar "python main.py", OpenManus se iniciara y te mostrara una interfaz de chat en la propia terminal. Desde ahi puedes escribirle tareas en lenguaje natural, igual que harías con Manus AI. Por ejemplo, puedes pedirle: "Busca informacion sobre el clima en Madrid esta semana y crea un resumen", o "Escribe un script de Python que organice mis fotos por fecha". El agente planificara la tarea, usara las herramientas disponibles (navegacion web, ejecucion de codigo, creacion de archivos) y te entregara el resultado.

Modo 2: Interfaz web (recomendado)

Si prefieres una interfaz grafica mas comoda en tu navegador, ejecuta:

```
cd %USERPROFILE%\OpenManus
venv\Scripts\activate
python web_run.py
```

Esto abrira una interfaz web en tu navegador en la direccion <http://localhost:8000> (o similar). Desde ahi podras interactuar con el agente de forma visual, ver los resultados formateados, y gestionar las conversaciones de manera mas intuitiva. Este modo es especialmente util si no te sientes comodo con la terminal.

Modo 3: Herramientas MCP (avanzado)

Si quieres usar herramientas MCP (Model Context Protocol) adicionales:

```
cd %USERPROFILE%\OpenManus
venv\Scripts\activate
python run_mcp.py
```

Las herramientas MCP permiten al agente conectar con servicios externos como bases de datos, APIs, y otros sistemas de forma mas avanzada. Este modo es para usuarios con experiencia. Tambien existe un modo multi-agente experimental con "python run_flow.py", pero es inestable.

5.2 Primera ejecucion y prueba

La primera vez que ejecutes OpenManus, es posible que tarde un poco mas de lo habitual en responder. Esto es normal: Ollama necesita cargar el modelo de IA en la RAM, lo cual puede tardar entre 30 segundos y 2 minutos. Las siguientes respuestas seran mas rapidas. Tambien es normal que la primera respuesta del agente tarde mas que las posteriores, ya que el modelo necesita "calentarse".

Ejemplo de tarea para probar:

```
Hola, puedes crear un archivo de texto con los numeros del 1 al 10, uno por linea, y guardarlo como numeros.txt?
```

Si OpenManus responde correctamente y crea el archivo, la instalacion ha sido un exito. A partir de aqui, puedes pedirle tareas cada vez mas complejas: buscar informacion en internet, escribir codigo, analizar datos, crear documentos, y mucho mas.

6. Solucion de problemas frecuentes

Es normal que durante la instalacion o el primer uso aparezcan algunos errores. La mayoría tienen solucion facil. A continuacion se recogen los problemas mas comunes que enfrentan los usuarios de OpenManus en Windows, junto con sus soluciones paso a paso. Si encuentras un error que no esta en esta lista, copia el mensaje de error exacto y buscalo en los "Issues" del repositorio de GitHub de OpenManus, donde la comunidad suele publicar soluciones rapidas.

6.1 Error: "python no se reconoce como comando"

Este error significa que Python no se agrego al PATH del sistema durante la instalacion. Para solucionarlo, tienes dos opciones. La primera es desinstalar Python y volver a instalarlo, esta vez marcando la casilla "Add Python to PATH" en la primera pantalla del instalador. La segunda opcion es agregar Python al PATH manualmente: abre "Editar las variables de entorno del sistema" en el menu Inicio, haz clic en "Variables de entorno", selecciona "Path" en la lista, haz clic en "Editar", y agrega las rutas "C:\Python311" y "C:\Python311\Scripts". Despues cierra y vuelve a abrir la terminal para que los cambios surtan efecto.

6.2 Error: "ollama no se reconoce como comando"

Este error indica que Ollama no esta en el PATH del sistema. Normalmente, Ollama se instala como una aplicacion de usuario en "C:\Users\TuNombre\AppData\Local\Programs\Ollama". Abre una nueva terminal y prueba de nuevo. Si aun no funciona, reinicia el ordenador. Si el problema persiste, agrega la ruta de Ollama al PATH manualmente, igual que se explica en el apartado anterior para Python.

6.3 Error: "No module named xxx"

Este error significa que falta alguna dependencia de Python. Asegurate de que el entorno virtual esta activado (debes ver "(venv)" en la linea de comandos) y vuelve a ejecutar el comando de instalacion de dependencias: "pip install -r requirements.txt". Si el error persiste con un modulo especifico, instálalo manualmente con "pip install nombre-del-modulo".

6.4 Error: "Connection refused" al conectar con Ollama

Este error significa que Ollama no esta ejecutandose. Abre una terminal y escribe "ollama serve" para iniciarlo. En otra terminal (sin cerrar la primera), ejecuta OpenManus. Ollama debe estar corriendo en segundo plano todo el tiempo que uses OpenManus. Si quieres que Ollama se inicie automaticamente con Windows, puedes agregarlo a la carpeta de Inicio del menu Inicio.

6.5 El modelo responde muy lento

Si el modelo tarda mas de 30 segundos en responder, es probable que tu CPU este cargando con el trabajo de ejecutar el modelo de IA. La solucion mas rapida es cambiar a un modelo mas pequeno. Abre una terminal y ejecuta: "ollama pull qwen2.5:3b". Luego edita el archivo config.toml y cambia "qwen2.5:7b" por "qwen2.5:3b". El modelo de 3B es menos inteligente pero significativamente mas rapido. Si tienes GPU NVIDIA con drivers CUDA instalados, Ollama la usara automaticamente y la velocidad mejorara mucho sin necesidad de cambiar de modelo. Tambien puedes probar "qwen3:8b" como alternativa mas reciente, aunque ten en cuenta que sus tool calls pueden fallar con mas frecuencia en Ollama.

6.6 Error en browser_use o navegacion web

Algunas funciones de navegacion web de OpenManus pueden dar errores en Windows si no tienes Google Chrome o Chromium instalado. Para solucionarlo, instala Google Chrome desde google.com/chrome si aun no lo tienes. OpenManus lo detectara automaticamente. Si el error es de version del paquete "browser-use", prueba a instalar una version compatible con: "pip uninstall browser-use && pip install browser-use==0.1.40".

7. Modelo alternativo: Agent-Zero sin Docker

Si despues de intentar instalar OpenManus encuentras que no se adapta a tus necesidades, existe una alternativa que tambien funciona sin Docker: Agent-Zero. Es un agente de IA autonomo de codigo abierto que puede instalarse directamente con Python y que soporta Ollama como motor de IA local, al igual que OpenManus. Su enfoque es ligeramente diferente: mientras que OpenManus esta disenado como un clon de Manus, Agent-Zero es mas un framework de agentes autonomos con memoria persistente y la capacidad de crear sub-agentes que colaboran entre si.

7.1 Instalacion rapida de Agent-Zero

La instalacion de Agent-Zero sin Docker sigue un proceso similar al de OpenManus. Necesitas tener Python 3.11 y Ollama instalados (las Fases 1 y 2 de esta guia son las mismas). Los pasos especificos de Agent-Zero son los siguientes:

```
cd %USERPROFILE%
git clone https://github.com/agent0ai/agent-zero.git
cd agent-zero
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Despues, para configurar Agent-Zero con Ollama, necesitas crear o editar el archivo de configuracion dentro de la carpeta del proyecto y apuntar el modelo a Ollama con la URL <http://localhost:11434> y el modelo qwen2.5:7b. La interfaz web de Agent-Zero se inicia con "python run_ui.py" y se accede desde el navegador en <http://localhost:50001>. Desde ahi puedes configurar el proveedor de modelo como Ollama directamente desde la interfaz grafica, sin editar archivos manualmente.

Aviso de seguridad: Ejecutar Agent-Zero sin Docker elimina el aislamiento de seguridad. El agente podra acceder directamente a tu sistema de archivos y ejecutar comandos. Esto es conveniente pero conlleva riesgos. Si un agente autonomo ejecuta un comando equivocado, puede borrar archivos importantes. Siempre revisa los comandos que el agente planea ejecutar antes de autorizarlos. OpenManus, por su parte, tiene un sistema de sandbox mas limitado que reduce este riesgo.

8. Resumen de todos los comandos

A continuacion se presenta un resumen completo de todos los comandos necesarios para la instalacion y ejecucion de OpenManus, ordenados secuencialmente. Puedes usar esta seccion como referencia rapida una vez que hayas completado la instalacion por primera vez. Los comandos estan pensados para ejecutarse en una terminal de Windows (cmd o PowerShell) en el orden indicado.

8.1 Instalacion completa (solo la primera vez)

```
# 1. Descargar e instalar Python 3.11 desde python.org
# (Marcar "Add Python to PATH")

# 2. Descargar e instalar Ollama desde ollama.com/download/windows

# 3. Descargar el modelo de IA
ollama pull qwen2.5:7b

# 4. Descargar e instalar Git desde git-scm.com/download/win

# 5. Clonar OpenManus
cd %USERPROFILE%
git clone https://github.com/FoundationAgents/OpenManus.git
cd OpenManus

# 6. Crear entorno virtual
python -m venv venv
venv\Scripts\activate

# 7. Instalar dependencias
pip install -r requirements.txt

# 8. Configurar Ollama
cd config
copy config.example-model-ollama.toml config.toml
cd ..
```

8.2 Ejecutar OpenManus (cada vez que lo uses)

```
cd %USERPROFILE%\OpenManus
venv\Scripts\activate
python main.py
```

8.3 Actualizar OpenManus (periodicamente)

```
cd %USERPROFILE%\OpenManus
venv\Scripts\activate
git pull
pip install -r requirements.txt
```

Recuerda: Ollama debe estar ejecutandose en segundo plano antes de iniciar OpenManus. Si cerraste el ordenador o reiniciaste, abre una terminal y escribe "ollama serve" antes de ejecutar "python main.py". En la mayoría de los casos, Ollama se inicia automáticamente con Windows.